

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



IMPLEMENTIERUNG

Techniken und Werkzeuge zur technischen Konstruktion von Software

Themen in dieser Vorlesung



Lernziele

- Verstehen, welche Fragen man sich zur zur effizienten und leistungsfähigen Organisation der Umsetzung von Anforderungen in Code stellen muss
- Wissen, welche Aktivitäten direkt zur Implementierung gehören oder in ihrem Umfeld liegen und welche Werkzeuge man dafür einsetzt



Implementierung

Wo stehen wir?

- Anforderungen sind erhoben (teilweise oder vollständig)
- Entwurf ist mit Architektur und ersten groben Klassenentwürfen erledigt etwa der Geschäftslogik,
- Mit Architektur wurden **wichtige Komponenten** ausgewählt: Z.B. Datenbank, Applikationsserver; **Programmiersprachen** wurden festgelegt
- Je nach Prozessmodell alles auf unterschiedliche Weise, ganz oder nur für einzelne Abschnitte, unterschiedlich detailliert
 - Wir vereinfachen, nutzen allgemeine logischen Reihenfolge: Anforderungserhebung – Entwurf – Implementierung
- Also folgt jetzt: Die **Implementierung** – der Code entsteht

Implementierung: Aktivitäten (1/2)

Gesteuert durch Vorgehensmodell:

- Konstruktion **eigenen Materials**, etwa
 - Klassen,
 - Dateien,
 - andere Ressourcen (Grafiken, Scripte, ...),
 - die zu Schichten / Modulen / Komponenten gehören
- Einsatz **fremden Materials**, etwa
 - Bibliotheken,
 - Frameworks,
 - Systemen (Datenbanken, Betriebssystemen, ...)
 - Diensten (extern, intern, lokal, entfernt)
- Material zu **Prozesssteuerung und -dokumentation** (etwa Aufgaben, Protokolle)

Implementierung: Aktivitäten (2/2)

Die entstehenden und eingesetzten Artefakte...

- ...müssen **getestet** werden
- ...entstehen schrittweise, werden weiterentwickelt: haben **Versionen** mit unterschiedlichen Voraussetzungen und Leistungen
- ...hängen voneinander ab
- ...müssen probeweise zusammengesetzt (**integriert**) werden
- ...als ganzer **Zwischenstand** ausgeliefert werden

Implementierung: Organisation der Aktivitäten

Zur effizienten und effektiven Durchführung dieser Aktivitäten

- durch viele Personen (unterschiedliche Erfahrung!)
- zu verschiedenen Zeitpunkten
- an mehreren Orten

sind **Festlegungen** und **Werkzeuge** nötig:

- Organisation und Ablage des Materials
- Werkzeugeinsatz – welche, wann, wie?

Implementierung:

Kann das nicht jeder alleine klären?

- Einmal festlegen **spart Aufwand**: Sonst muss jeder einzelne sich es selbst überlegen
- Zentrale Festlegungen **reduzieren Fehler aus Unerfahrenheit**
- Einheitlichkeit...
 - ...**erleichtert Zusammenarbeit**, jeder findet sich schneller (oder überhaupt) zurecht
 - ...**erleichtert Werkzeugeinsatz** – oder ermöglicht ihn erst
- Teilweise geht es schlicht um Prozesse – deren **Optimierung auf Effizienz und Qualität** erfordert Gesamtbetrachtung und Einheitlichkeit

Implementierung:

Was ist also zu tun?

- Auswahl Basiswerkzeuge Entwicklungsplattform (IDE)
- Konventionen für Namen und Projektorganisation
- Organisation der gemeinsamen Arbeit auf Projektressourcen (Dateien) mit Versionierung und Dokumentation
- Organisation zum Umgang mit Abhängigkeiten
- Organisation des „Builds“ (d.h. Zusammenbaus des Systems)
- Organisation des Change- und Bugtrackings
- Prüfung und Durchsetzung aller festgelegten Standards
- Prüfung und Optimierung von Qualität

Oft entscheidend: Nicht Inhalt konkreter Regelung, sondern Vorliegen einer Regelung.

Wichtig: Möglichst weitgehende Automatisierung.

Entwicklungsplattform: Werkzeuge

- Ein **syntax-gesteuerter Editor** mit integriertem **Compiler** um Code zu erstellen, editieren und zu übersetzen
- Einen **Debugger** für die Programmiersprache
- **Graphische Editoren** etwa für UML-Modelle
- **Test-Werkzeuge** wie JUnit die automatisch Tests auf einer neuen Version ausführen können
- **Projektunterstützungswerkzeuge** die dabei unterstützen, Code aus verschiedenen Projekten zu organisieren

Integrierte Entwicklungsumgebungen (IDEs)

- Werkzeuge der Entwicklungsplattform werden häufig in einer integrierten Entwicklungsumgebung kombiniert (Integrated Development Environment, IDE)
- IDE unterstützt verschiedene Aspekte der Softwareentwicklung innerhalb eines **einheitlichen Rahmenwerks** und mit **einheitlicher Oberfläche**
- IDEs werden auf **spezifische Programmiersprachen** ausgerichtet: Entweder spezifische Entwicklung für eine Sprache oder Instanziierung allgemeiner IDE mit sprach-spezifischen Erweiterungen sein.

Namen sind wichtig!

Bei Softwareentwicklung **vergeben wir ständig Namen**: Variablen, Funktionen, Argumente, Klassen, Pakete, Dateien, Verzeichnisse.

- **Gute Namen helfen**, sich schnell auch in einem fremden Softwaresystem orientieren zu können und ohne weitere Details ein Verständnis davon zu bekommen, womit man es zu tun hat
- **Umgekehrt**: Findet man keinen passenden Namen, dann hat man nicht verstanden, was man gerade erzeugt – oder man erzeugt das falsche

Namen sind wichtig!

Beispiel: Was macht dieser Quelltext?

```
public List<int[]> getThem() {  
    List<int[]> list1 = new ArrayList<int[]>();  
    for (int[] x : theList)  
        if (x[0] == 4)  
            list1.add(x);  
    return list1;  
}
```



```
public List<int[]> getFlaggedCells() {  
    List<int[]> flaggedCells = new ArrayList<int[]>();  
    for (int[] cell : gameBoard)  
        if (cell[STATUS_VALUE] == FLAGGED)  
            flaggedCells.add(cell);  
    return flaggedCells;  
}
```



Aus Robert C. Martin: Clean Code, 2008.

Namen sind wichtig!

Beispiel: Was macht dieser Quelltext?

Noch ein Schritt weiter verbessert:

```
public List<Cell> getFlaggedCells() {  
    List<Cell> flaggedCells = new ArrayList<Cell>();  
    for (Cell cell : gameBoard)  
        if (cell.isFlagged())  
            flaggedCells.add(cell);  
    return flaggedCells;  
}
```



Aus Robert C. Martin: Clean Code, 2008.

Hier anstatt Verwendung von `int[]` eine Klasse `Cell` geschrieben, die absichtsenthüllend benannte Funktion `isFlagged()` enthält.

Namen und Namenskonventionen

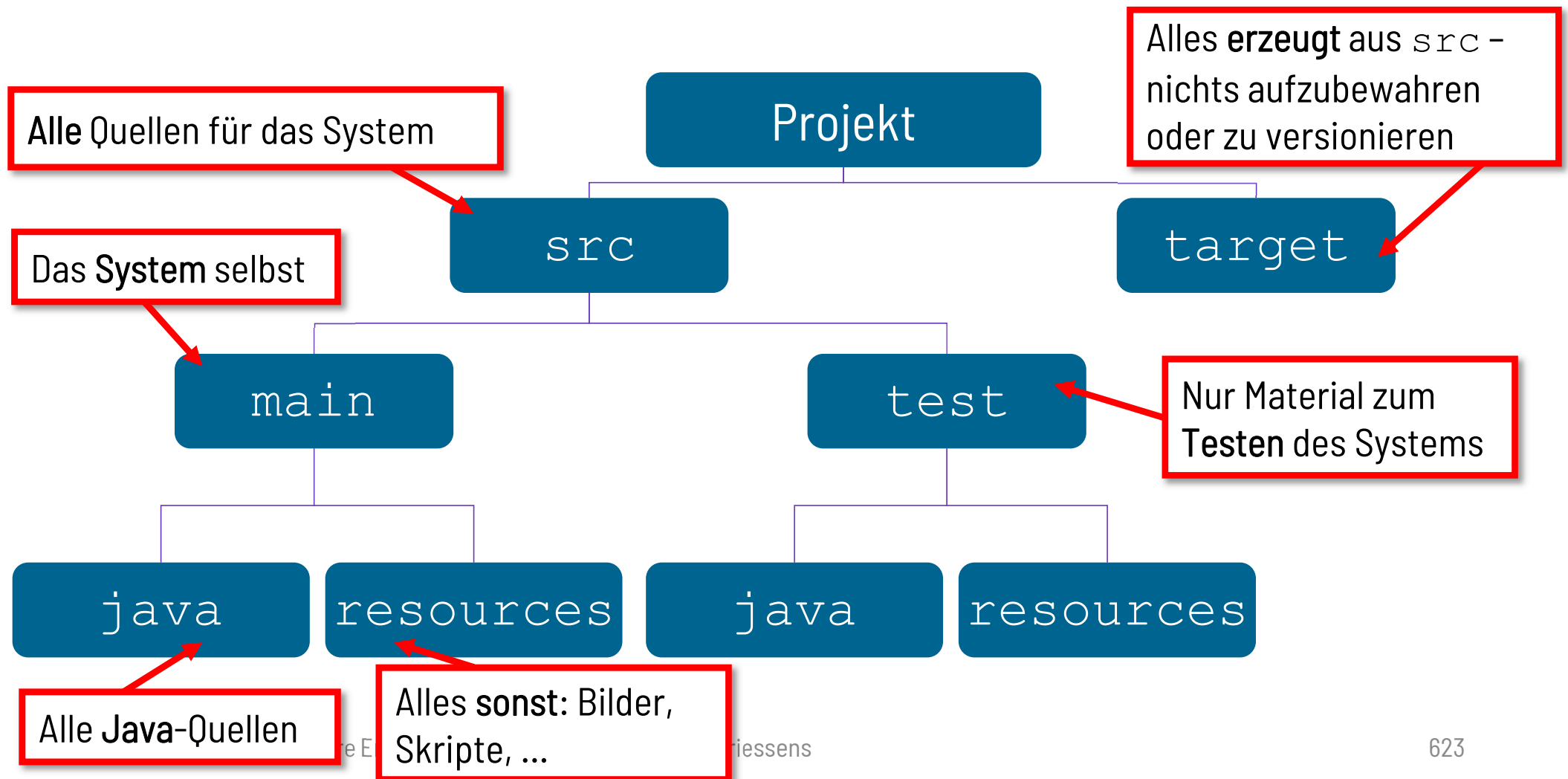
Namen sollten überlegt vergeben werden:

- Absichtsenthüllende Namen
- Fehlinformation vermeiden
- Mit Namen Unterschiede ausdrücken
- Suchbare Namen verwenden
- Einheitliche Namen ohne Wortspiele (wenn Sie witzig sein wollen, dann nicht hier)
- Für Klassen: Substantive. Für Methoden: Verben.

Plus: Namenskonventionen! Sie helfen, Namen schneller zu finden und über einheitliche Namen schneller die Art des „Dings“ zu erkennen.

Ablage der Implementierung

Häufig verwendete Java-Projektstruktur:



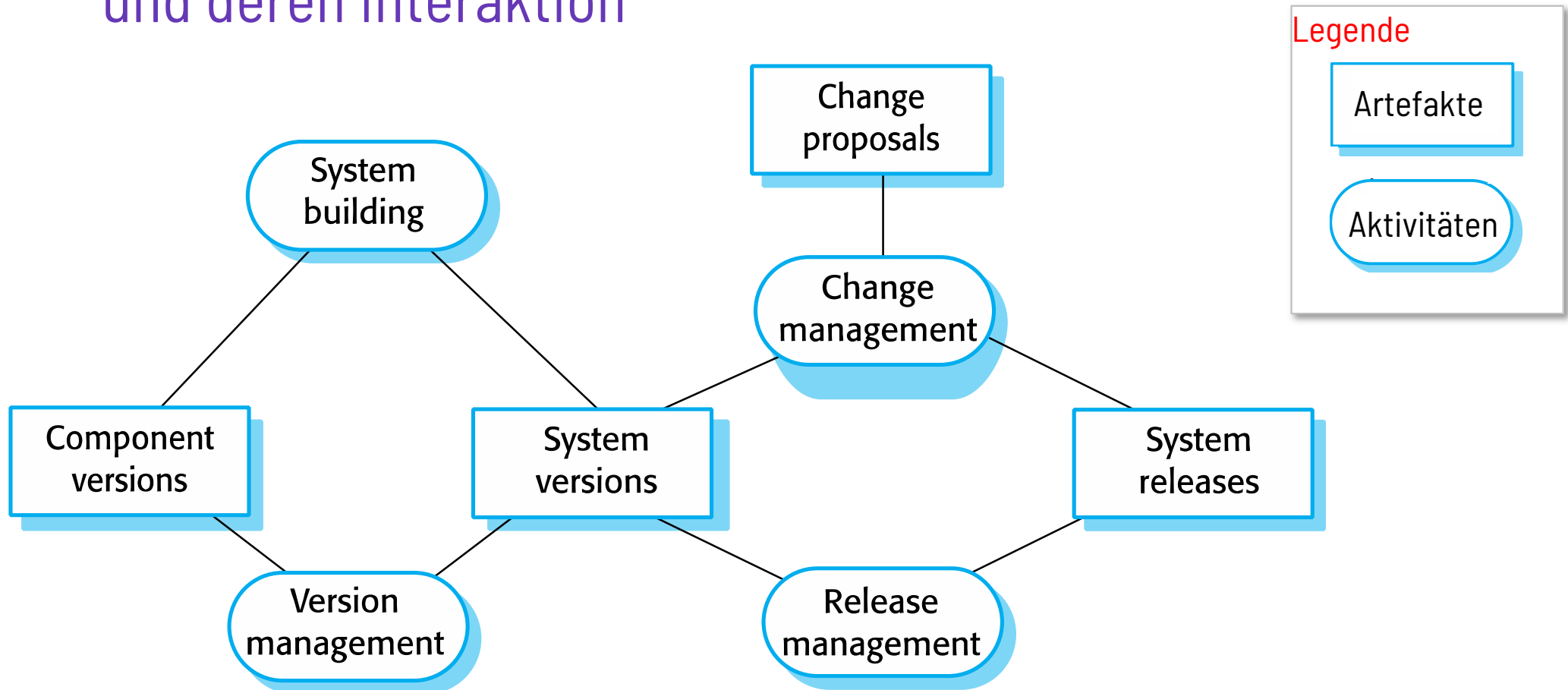
Ablage der Implementierung: Projektstruktur

- Projektstruktur sollte passen zu
 - Projekttyp
 - Programmiersprachen
 - verwendeten Werkzeugen
- **Projektvorlagen** für unterschiedliche Projekttypen verfügbar, etwa in
 - Integrierte Entwicklungsumgebungen (wie *Eclipse*)
 - Werkzeugen zur Organisation von Projekt und Build (wie *Maven*)

Konfigurationsmanagement (Configuration Management, CM)

- Allgemeiner Prozess des Managements eines sich ändernden Softwaresystems
- Ziel: Unterstützung des Systemkonstruktions- und -integrationsprozesses so, dass alle Entwickler...
 - ...Quelltext und Dokumente auf gesteuerte Weise zugreifen können,
 - ...herausfinden können, welche Änderungen in welcher Version wann vorgenommen wurden, und
 - ...Komponenten zur Konstruktion des Systems erweitern und hinzufügen können.

Vier Aktivitäten des Konfigurationsmanagements und deren Interaktion



Wir vertiefen die zentralen Bereiche

- Versionsmanagement
- System building (Build Management)

Dokumentation

- Dokumentation gilt als ewiges Sorgenkind der Software-Entwicklung
- **Dokumentieren ist beständige Daueraufgabe**
- Nachträgliche Dokumentation ist unzureichende Notlösung – die Information, die es aufzuzeichnen gilt, ist vergessen oder die Person mit dieser Information im Kopf verschwunden
- Dokumentation tritt demnach im Software-Lebenslauf nicht als eigene Tätigkeit auf.
- **Software-Entwickeln ist Dokumentieren!**

Dokumentations-Kategorien im Überblick

Dokumenten-kategorie	Zweck	Adressaten	Aktuali-sierung	Aufbewahrungsfrist
System-dokumen-tation	Erhaltung und Transfer des Know-hows, Einsatz des Systems	Entwickler, Wartungsingenieure, Tester, Kunden	erforderlich	bis Ende der Systemnutzung
Projekt-dokumen-tation	Führung des Projekts, Analyse nach Projektende	Linien- und -Projektmanager, Kunden	unerwünscht	bis Projektende plus n Jahre
Qualitäts-dokumen-tation	Nachweis der Maßnahmen zur Qualitätssicherung	Linien- und -Projektmanager, Kunden	verboten	bis Ende der Systemnutzung
Prozess-dokumen-tation	Beschreibung der Vorgehensweise, Unterstützung bei der Durchführung	alle, die aktiv am Projekt beteiligt sind	sinnvoll	bis Ende der Systemnutzung und so lange, wie der Hersteller existiert

Integrierte Dokumentation

- Integrierte Dokumentation (etwa Javadoc) ist **leichter zu bearbeiten** und hat bessere Chancen, **nachgeführt** zu werden
- **Sie reicht aber in keinem Falle aus!**
- Bei systematischem Vorgehen sind mindestens
 - 40 % des Aufwands geleistet und damit auch
 - 40 % der Software entstanden, **bevor Code geschrieben** wird
- Begriffslexikon, Spezifikation und Architektur-Entwurf müssen unbedingt vorher entstehen!
 - Können nicht in Code-Komponenten abgelegt werden, wie auch Projektdokumentation

Ansätze integrierter Dokumentation

Beispiele integrierter Dokumentation:

- **Javadoc:**
 - Annotation von Java-Quelltext
 - Erzeugung etwa von HTML-Dokumenten direkt aus Java-Quelltext
- **Modellgetriebene Softwareentwicklung (MDSD):**
 - Formalisierung von Anforderungsdokumenten
 - Dann Einsatz zur automatischen Erzeugung von Systemteilen
 - Damit: Beschreibung der Anforderung ist gleichzeitig auch Teil der Programmierung und Dokumentation

Separate Dokumentation

Die separate Dokumentation ist grundsätzlich gefährdet, sie kann nur funktionieren, wenn

- die **Anforderungen** an die Dokumente und die **Verantwortlichkeiten** für das Erstellen, Prüfen und Freigeben klar sind,
- Dokumentation hoch **bewertet** und **anerkannt** wird,
- das **Inhaltsverzeichnis** (die Struktur) der Dokumente zu Beginn festgelegt ist,
- **Werkzeugunterstützung** verfügbar ist,
- jedes Dokument nach Fertigstellung und nach Änderungen **geprüft** wird und
- alle Dokumente einer effektiven **Konfigurationsverwaltung** unterstellt werden.

Folgen schlechter Dokumentation

- In Praxis: Dokumentation gehört meist zu **Problemzonen** des Software Engineerings – warum?
 - Software-Entwickler haben Dokumentieren **weder in der Ausbildung noch in der Praxis gelernt**, sie können es nicht
 - Priorität 1: Einhaltung des Liefertermins, auch Priorität 2 und 3. Da die Termine in fast allen Projekten so vorgegeben sind und Dokumentation oft nicht eingeplant wird, wird nicht dokumentiert. **Dazu fehlt einfach die Zeit.**
- Folgen:
 - Wartungsingenieure arbeiten als Archäologen.
 - Grundlagen für die Handbücher fehlen
 - Retrospektive Analysen sind nicht möglich. Damit hat die Organisation keine Möglichkeit, aus dem Projekt zu lernen.

Ausblick

Wir werden weiter vertiefen:

- Versionsmanagement mit Git
- Build-Management und Projektorganisation mit Maven
- Softwarequalität, ihre konstruktive Sicherung und werkzeuggestützte Überwachung
 - Testen von Software

Ausbauthemen (-> Software Engineering-Praktikum):

- Continuous Integration mit Jenkins

Zusammenfassung

- Zur Implementierung gehört mehr als die reine Erstellung von Programmcode
- Werkzeuge insbesondere zur Organisation der Zusammenarbeit vieler Personen in einem Projekt sind ebenso zentral wie Werkzeuge zur effizienten Umsetzung einzelner Tätigkeiten und zur Komplexitätsreduzierung

